Buna Market

Online shop for Ethiopian coffee, spices and crafts.
Frontend + backend. 8 weeks.

TEAM

Gizachew Dagnew Fetene / lead
Beimnet Tariku Feyissa
Genet Tilahun
Habtamu Dires Bedilu

SCOPE

Next.js + TypeScript
Express + TypeScript
PostgreSQL + Chapa

github.com/Zemenaytech/buna-market

03

QUESTIONS?
TELEGRAM @Kant_III

01 MISSION

Build an online shop for Ethiopian coffee, spices and handmade goods: buna beans, berbere, mitmita, jebena pots, mesob baskets and netela scarves. Shoppers browse, add to cart and pay with Chapa (Ethiopia's payment gateway) in test mode. Admins manage products and orders.

At the end of week 8, Buna Market is live: a shopper can find a product, pay with a Chapa test card, and see the order in their history. Buna Market is a working name; the team can rename it.

KEY RULE

Chapa test keys only. Real keys never go in the repo, in chat, or in a screenshot. Secrets live in `.env` files that are in `.gitignore`.

02 TEAM

NAME	ROLE	OWNS
Gizachew Dagnew Fetene `@Gech3734d`	Team lead, full-stack	Repo setup, auth, cart API, checkout UI, code reviews
Beimnet Tariku Feyissa `@beimnet06`	Backend	Products API, search, orders, Chapa payments, recommendations
Genet Tilahun `@Edeledele`	Backend + admin	Reviews and ratings API, admin API, admin dashboard pages
Habtamu Dires Bedilu `@habtamudires4-ctrl`	Frontend	Shop home, category and product pages, cart UI, order history

Gizachew has Maintain access and runs the Friday standup. Everyone else has Write access. Habtamu pairs with Gizachew on the first shop page, then builds the next pages on his own.

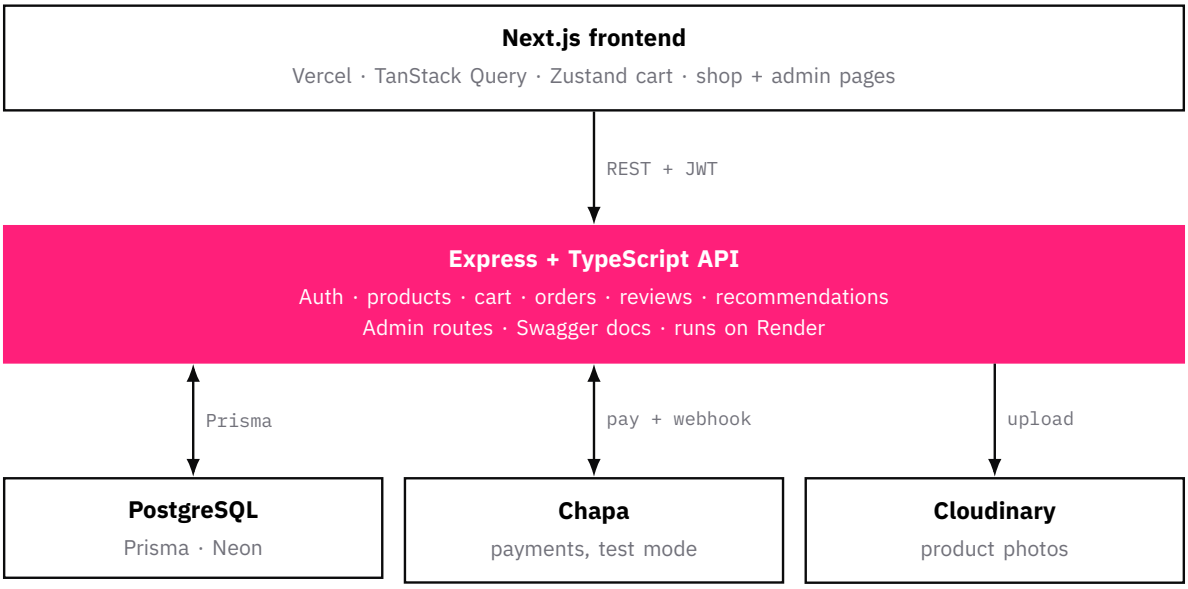
REPO

`github.com/Zemenaytech/buna-market`
Private repo. One repo, two folders: `web/` (Next.js) and `api/` (Express), plus `docker-compose.yml` for PostgreSQL. Accept the invite in your email or on github.com/notifications; it expires after 7 days.

03 STACK

PART	TOOL	WHY
Frontend	Next.js (App Router) + TypeScript + Tailwind + shadcn/ui	Same base as the other teams
Server data	TanStack Query	Caching, loading and error states
Cart state	Zustand	Small and simple; saved to the API when logged in
Backend	Node.js (LTS) + Express + TypeScript	The team already knows Express
Database	PostgreSQL + Prisma	Typed queries and easy migrations
Validation	Zod	Same schemas checked on the API and in forms
Auth	JWT access token + refresh token in an httpOnly cookie	Standard, safe login
Images	Cloudinary (free tier)	Upload and resize product photos
Payments	Chapa test mode	Local payment gateway, test keys only
API docs	Swagger (swagger-ui-express)	Frontend can try every endpoint
Tests	Vitest + Supertest	Test every API route
Local setup	Docker Compose for PostgreSQL	Same database for everyone
Deploy	Vercel (web), Render (API), Neon (database)	Free tiers

04 ARCHITECTURE



The frontend never talks to the database or to Chapa directly. Every read and write goes through the API, and the API checks the login first.

05 FEATURES AND OWNERS

FEATURE	OWNER	NOTES
Repo setup, auth (sign up, log in, refresh)	Gizachew	Done in sprint 1 so others can build on it
Products and categories API, search	Beimnet	Filter by category and price; paginate results
Shop home, category page, product page	Habtamu	Pairs with Gizachew on the first page
Cart page and cart API	Habtamu (UI), Gizachew (API)	Works as a guest too, then merges on login
Checkout, orders API, Chapa payment	Gizachew (UI), Beimnet (API)	An order is paid only after Chapa's webhook confirms it
Reviews and ratings	Genet	API plus the stars UI on the product page
Admin dashboard	Genet	Add and edit products, upload photos, change order status
"You may also like"	Beimnet	Start with same category and "bought together" counts in SQL
Order history and profile	Habtamu	Simple pages that read the API

STRETCH GOAL: SMART SEARCH

Beimnet adds search by meaning, so "something for a coffee ceremony" finds the jebena, cups and incense. Start simple, measure if it helps, then improve.

MAIN API ENDPOINTS

```
POST /api/auth/register|login|refresh   GET  /api/products?category=&q=&page=
GET  /api/products/:slug                 GET  /api/categories
GET  /api/cart                           POST /api/cart/items
POST /api/orders                         GET  /api/orders/me
POST /api/payments/chapa/webhook         POST /api/products/:id/reviews
     /api/admin/*   (admin only)
```

MONEY RULE

Store prices as whole santim (integers), never floats. 250.00 ETB is stored as `25000`. Format to birr only when you show it on screen.

06 SPRINT PLAN

S01 / WEEKS 1–2	S02 / WEEKS 3–4	S03 / WEEKS 5–6	S04 / WEEKS 7–8
Figma designs Repo + Docker Auth API + login pages Products API + seed data	Shop + product pages Cart (guest + login) Admin: add products	Checkout + Chapa test Reviews and ratings Order history pages	You may also like Deploy, bug fixes **Launch**
DEMO FRIDAY	DEMO FRIDAY	DEMO FRIDAY	**LAUNCH + FINAL DEMO**

Anything not finished moves to the next sprint. The week 8 launch date stays fixed.

07 HOW WE WORK

All three teams follow the same rules, so anyone can read and help on any repo.

SHARED TOOLS

- **Code:** one private GitHub repo per project, under the `Zemenaytech` organization.
- **Tasks:** a GitHub Projects board per team with four columns: Backlog, This sprint, In review, Done.
- **Design:** one Figma file per team. Design a page before you build it.
- **Chat:** one channel per team, plus a shared channel for all interns.

GIT AND PULL REQUESTS

01 Never push to `main`. Create a branch per task: `feature/listing-filters`, `fix/cart-total`.
02 Keep pull requests small, one task each. Aim for under 400 changed lines.
03 Write the PR description: what you changed, why, how to test it, and a screenshot for UI work.
04 Every PR needs one approval before merge. Team leads review their team; the mentor reviews leads.
05 Use clear commit messages: `feat: add price filter`, `fix: wrong total on cart page`.
06 Squash and merge. Delete the branch after.

MEETINGS

All meetings are on Friday. The rest of the week is for building.

MEETING	WHEN	WHAT HAPPENS
Standup	Every Friday, 30 min	What I did this week, what I will do next week, what is blocking me
Demo	Every 2nd Friday, after standup	Each team shows working features to everyone
Sprint planning	Every 2nd Friday, after demo	Pick tasks from the backlog for the next two weeks

STUCK?

Stuck for more than 2 hours? Ask in your team channel first. Still stuck, or something is not working out? Contact the program lead directly on Telegram: **@Kant_III**. Asking early is part of the job, not a weakness.

DEFINITION OF DONE

A task is done only when every box is ticked:

- ☐ It works on desktop and on a phone screen.
- ☐ Lint passes and there are no console errors.
- ☐ It has tests where the team set a rule for tests (backend logic always).
- ☐ The PR is reviewed, approved and merged.
- ☐ It is live on the staging deploy.

08 WEEK 1 CHECKLIST

Finish this list before the first sprint planning.

- ☐ Install Git, VS Code, Node.js LTS and Docker Desktop.
- ☐ Accept the GitHub invite and clone `github.com/Zemenaytech/buna-market`.
- ☐ Run the project on your laptop and open a first PR that adds your name to the README.
- ☐ Read this guide twice. Write down your questions for the Friday standup.
- ☐ Habtamu and Gizachew: do the Next.js “Learn” course.
- ☐ Beimnet and Genet: do the Prisma quickstart and read the Chapa docs.
- ☐ Gizachew: set up `web/` and `api/`, lint and formatting, Docker Compose and the board.
- ☐ Together: agree on the sprint 1 page designs in Figma.

09 LEARN FROM

TOPIC	LINK
Next.js course	`nextjs.org/learn`
Express	`expressjs.com`
Prisma	`prisma.io/docs`
Zod	`zod.dev`
Chapa developer docs	`developer.chapa.co`
Cloudinary	`cloudinary.com/documentation`
Vitest	`vitest.dev`